

CircuAudit AI

An Autonomous Multi-Agent Circular Packaging & Material Audit Engine engineered for the Indian Waste Ecosystem and Extended Producer Responsibility (EPR) Compliance.

TARGET MARKET

India FMCG & Industrial Recycling

TECHNICAL STACK

LangGraph, YOLOv8, Gemini Multi-Modal

DOCUMENT VERSION

v1.0.0 (Production-Ready)

1. Executive Summary & Market Fit

In India, the Central Pollution Control Board (CPCB) enforces strict **Extended Producer Responsibility (EPR)** regulations. These mandates compel Fast-Moving Consumer Goods (FMCG) conglomerates to ensure their post-consumer packaging is verifiably recycled. Despite spending massive sums on regulatory compliance certificates, brands possess zero visibility into their products' actual end-of-life status. Millions of tons of plastic flow through unorganized waste streams and are rejected at sorting centers due to poor material composition, non-scannable pigments, or complex multi-layer designs.

CircuAudit AI solves this visibility gap. It is a multi-agent framework that continuously monitors and processes physical waste streams using low-cost edge vision systems, cross-references anomalies against regional environmental laws, and uses generative AI to engineer compliant, circular packaging alternatives.

THE VALUE PROPOSITION

This architecture transitions recycling tracking from a lagging statistical estimation to a real-time, asset-level data audit. It allows developers to deploy high-value B2B SaaS portals for FMCG enterprises while significantly increasing the efficiency of commercial sorting operations.

2. Stakeholder Value Analysis

The platform creates an active, financially incentivized feedback loop across three core sectors of the waste management lifecycle:

Stakeholder	Critical Pain Point	System Remediation & Value Delivery
FMCG Brands	High risk of CPCB non-compliance fines; zero visibility into packaging failure modes at sorting facilities.	Automated Material Auditing detects exactly which product lines fail sorting filters, providing actionable design modifications.
Recyclers & Sorters	Manual sorting bottlenecks; processing lines clogged by unrecyclable multi-layer plastics (MLP).	Automated prediction and high-speed edge classification allow processing facilities to sort high-value polymers efficiently.
Environmental Impact	Massive downcycling and landfill accumulation due to toxic inks, adhesives, and non-recyclable coloring.	Direct design feedback changes upstream manufacturing choices, accelerating authentic circularity.

3. Multi-Agent System Architecture

The platform operates on a stateful, multi-agent layout managed via deterministic state transitions. This prevents the traditional infinite-loop issues of open-ended agents.

Agent 1: Vision Classification & Inference Agent

Deployed on low-cost compute at aggregation points to process real-time frames. It isolates items, detects text/branding using OCR, determines material properties (e.g., PET vs. HDPE vs. MLP), and notes degradation states. High-confidence items pass through; anomalies trigger a high-fidelity cloud evaluation.

Agent 2: EPR Law & Compliance Critic Agent

An enterprise Retrieval-Augmented Generation (RAG) agent tied to a database containing up-to-date Plastic Waste Management Rules. It evaluates parsed packaging structures against national mandates, identifying compliance deltas and computing potential corporate penalty liabilities.

Agent 3: GenAI Material Redesign Engine

When a material structural failure is reported, this agent leverages multi-modal models to automatically construct alternative material specifications. It provides structural adjustments (e.g., substituting an adhesive, shifting ink chemistry, or changing color hex codes) to maximize sorting pass rates.

4. Production-Grade Technical Implementation

To ensure production readiness, the agent framework uses a stateful orchestration pattern. Below is a foundational implementation blueprint designed with Python and LangGraph.

```

import typing as t
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, END

# Define the state variables shared across the workflow
class AuditState(TypedDict):
    image_path: str
    brand_detected: str
    material_type: str
    is_compliant: bool
    compliance_notes: str
    redesign_blueprint: str

# Node 1: Vision Inference Engine
def classify_waste_material(state: AuditState) -> t.Dict[str, t.Any]:
    # Emulates multi-modal processing via Gemini Flash API
    return {
        "brand_detected": "BrandX Beverages",
        "material_type": "PET with Metallic Carbon-Black Dye"
    }

# Node 2: Compliance Cross-Reference
def verify_epr_compliance(state: AuditState) -> t.Dict[str, t.Any]:
    # Cross-references extracted data against India EPR Vector DB
    material = state["material_type"]
    if "Carbon-Black" in material:
        return {
            "is_compliant": False,
            "compliance_notes": "Carbon-black pigment prevents optical sorting systems."
        }
    return {"is_compliant": True, "compliance_notes": "Complies with active mandates."}

# Node 3: Generative Packaging Redesign
def generate_redesign_specs(state: AuditState) -> t.Dict[str, t.Any]:
    # Generates actionable structural packaging optimization proposals
    blueprint = (
        "RECOMMENDATION: Replace Carbon-Black dye with Infrared-transparent pigment."
        "BENEFIT: Raises local recycling facility detection accuracy from 12% to 94%."
    )
    return {"redesign_blueprint": blueprint}

# Routing logic based on compliance outcomes
def route_after_compliance(state: AuditState) -> str:
    if not state["is_compliant"]:
        return "generate_redesign"
    return END

# Initialize and orchestrate the state graph
workflow = StateGraph(AuditState)
workflow.add_node("classify_material", classify_waste_material)
workflow.add_node("check_compliance", verify_epr_compliance)
workflow.add_node("generate_redesign", generate_redesign_specs)

workflow.set_entry_point("classify_material")
workflow.add_edge("classify_material", "check_compliance")

```

```
workflow.add_conditional_edges(
    "check_compliance",
    route_after_compliance,
    {"generate_redesign": "generate_redesign", END: END}
)
workflow.add_edge("generate_redesign", END)
app = workflow.compile()
```

5. Scaling the System to Production

Deploying a reliable system in the field requires moving past basic API configurations to handle scaling costs and messy data environments effectively:

- **Hierarchical Inference Filters:** Avoid passing raw, high-resolution video streams directly to expensive cloud LLMs. Utilize an edge detector like **YOLOv8** on-site to crop relevant waste items, and pass only high-variance anomalies up to the heavy multi-modal endpoints.
- **Asynchronous Task Architectures:** Real-time sorting operations cannot wait for LLM generations. Decouple data tracking from generative reporting using an asynchronous task worker configuration powered by **Celery** and **Redis**.
- **State Evaluation and Observability:** Integrate agent-tracking platforms such as **LangSmith** to record step-by-step performance. This helps verify compliance accuracy, trace token consumption, and catch edge cases where packaging rules change.